

The artifact I selected for this enhancement is my CS-330 3D Scene Project from Computational Graphics and Visualization. I originally created this project during CS-330 using C++, OpenGL, GLFW, GLEW, and GLSL shaders to build an interactive 3D scene. The scene includes several objects such as a coffee mug, flashlight, dog tags, coin, and a textured tabletop. Users can move around the scene using keyboard and mouse controls to view the objects from different perspectives. I selected this artifact because it has provided opportunities throughout CS-499 to demonstrate different areas of computer science while continuing to build upon the same project. For this final enhancement, I focused specifically on database concepts and data management.

I selected this artifact for the database category because it gave me an opportunity to improve how data is stored and managed within the application. In the previous version of the project, information about scene objects such as the coin and dog tags was hardcoded directly inside the source code. While this worked for a small project, every change required modifying the code and recompiling the application.

For this enhancement, I created a sceneObjects.csv file that stores scene object records outside of the application. Each record contains information such as the object type, texture assignment, scale, rotation, position, and UV values. I also developed a LoadSceneObjectsFromCSV() function that reads the records from the CSV file, parses the data, and loads the information into SceneObject objects at runtime. This allowed me to remove hardcoded object definitions from the source code and instead load object data from an external source.

Although this enhancement uses a CSV file rather than a traditional relational database, it demonstrates many of the same concepts, including structured data storage, record retrieval, data

parsing, and separation of data from application logic. Before the enhancement, the coin and dog tag objects were created directly inside `PrepareScene()` using hardcoded values. After the enhancement, those values are stored externally and loaded dynamically when the application starts. This makes the project easier to maintain and provides a more scalable approach for adding or modifying scene objects in the future.

This enhancement supports the course outcome related to using well-founded techniques, skills, and tools in computing practices for implementing computer solutions that deliver value. By separating object data from application code, I improved the maintainability and flexibility of the project while demonstrating practical data management concepts. The enhancement also supports the outcome involving the design and evaluation of computing solutions. Instead of relying on hardcoded object definitions, the application now retrieves structured data from an external source and converts that data into usable objects within the scene.

One thing I have learned throughout this program is that many computer science concepts are connected. Earlier milestones focused on software design and organization, followed by data structures and algorithms. This enhancement brought those concepts together by showing how data can be stored externally, loaded into a data structure, and then processed by the application. My overall outcome coverage plan remains unchanged, and this enhancement completes the database portion of my ePortfolio while remaining separate from the previous milestones.

One of the biggest lessons I learned from this enhancement was the value of separating data from application logic. When I originally created this project in CS-330, hardcoding object information seemed like the simplest solution because there were only a few objects in the scene. However, as I continued enhancing the project throughout CS-499, it became clear that storing object information externally would make the project easier to maintain and expand.

The most challenging part of the enhancement was implementing the CSV loading process and making sure the data was parsed correctly. Since every object depends on multiple values such as position, rotation, scale, texture assignments, and UV settings, even a small formatting mistake could cause rendering problems or prevent objects from loading correctly. There were several times during testing when I had to troubleshoot file paths, object values, and parsing issues before everything worked as expected. While it was sometimes frustrating, it helped me gain a better understanding of how applications retrieve and process external data.

Another challenge was making sure this enhancement stayed focused on database concepts without overlapping with the software design and algorithms enhancements completed earlier in the course. Since I used the same artifact for all three categories, I needed to clearly separate each enhancement and ensure that each one demonstrated a different area of computer science. Overall, this enhancement strengthened my understanding of data management, record retrieval, and application design. It also reinforced the idea that maintaining data separately from application logic can make software easier to update, maintain, and expand over time.